

CASE STUDY 1

Selenium Web UI Automation

Item	Description
Project Type	Web UI Test Automation
Role	Software QA Engineer
Testing Type	Functional Testing, UI Testing, Regression Testing
Programming Language	Python
Automation Framework	Selenium WebDriver, PyTest
Design Pattern	Page Object Model (POM)
Tools	VS Code, Git, GitHub
Browser	Google Chrome
Project Status	Completed

Project Overview

This project focused on automating functional testing of a web application using **Python**, **Selenium WebDriver** and **PyTest**. The automation framework was developed to validate user interactions with various web elements while following industry best practices for test automation.

The project included automated verification of web forms, radio buttons, checkboxes, text fields, dropdown lists and navigation between web pages. Special attention was given to creating maintainable and reusable test scripts by applying the **Page Object Model (POM)** design pattern and synchronization techniques using **Explicit Waits**.

The primary goal was not only to automate manual test cases but also to build a scalable and well-structured automation framework that could be easily maintained and extended.

The project was developed as part of my practical training in software test automation and reflects my ability to design structured, maintainable and scalable automated testing solutions.

Project Objective

The objectives of this project were:

- Automate repetitive functional UI test scenarios.
- Validate user interactions with common web elements.
- Improve test reliability using Explicit Waits.
- Apply the Page Object Model (POM) design pattern.

- Develop reusable and maintainable test scripts.
 - Organize automated tests using the pytest framework.
 - Improve understanding of modern UI automation practices.
-

Test Environment

Component	Description
Programming Language	Python
Test Framework	PyTest
Automation Tool	Selenium WebDriver
Design Pattern	Page Object Model (POM)
IDE	Visual Studio Code
Browser	Google Chrome
Driver Management	WebDriver Manager
Version Control	Git & GitHub
Operating System	Windows 11

Test Scenarios

The automated test suite covered the following scenarios:

- Opening the Selenium Web Form application.
 - Validating page availability.
 - Locating web elements using ID, Name, XPath and CSS Selectors.
 - Entering data into text fields.
 - Selecting radio buttons.
 - Validating checkbox behavior.
 - Working with dropdown menus.
 - Clicking buttons and verifying expected results.
 - Synchronizing test execution using Explicit Waits.
 - Verifying expected application behavior through assertions.
-

Test Automation Approach

The automation framework was designed using the **Page Object Model (POM)** architecture to improve code readability, maintainability and reusability.

Reusable page classes were created to separate page locators from test logic. Test execution was managed using the **PyTest** framework, while **WebDriver Manager** automatically handled browser driver configuration.

To improve execution stability, **Explicit Waits (WebDriverWait)** and Selenium Expected Conditions were used instead of fixed delays.

Challenges & Solutions

During implementation several technical challenges were encountered.

One of the main challenges involved handling dynamic web elements and ensuring stable synchronization between page loading and test execution.

This was successfully addressed by implementing **WebDriverWait** together with Selenium **Expected Conditions**, significantly improving execution reliability.

Another challenge involved organizing reusable automation code. The **Page Object Model** design pattern was adopted to reduce code duplication and simplify future maintenance.

Results

The completed automation framework successfully demonstrated:

- Stable automated execution of UI test scenarios.
 - Successfully automated repetitive manual testing activities, reducing execution time and improving repeatability.
 - Reliable validation of user interface functionality.
 - Improved maintainability through modular framework design.
 - Reusable automation components.
 - Better understanding of automation architecture and testing best practices.
-

Lessons Learned

This project significantly strengthened my practical understanding of Selenium automation.

The most valuable lessons learned include:

- Reliable element locators are essential for stable automated tests.
- Explicit Waits provide significantly better synchronization than static waits.

- The Page Object Model simplifies framework maintenance and improves code organization.
 - Well-structured automation projects are easier to debug, maintain and extend.
 - Automation testing requires not only programming skills but also a systematic testing mindset.
 - Test automation is most effective when combined with thoughtful test design and maintainable architecture.
-

Technologies Used

Python • Selenium WebDriver • PyTest • WebDriver Manager • Page Object Model • Git • GitHub • Visual Studio Code • Google Chrome

Key Skills Demonstrated

Competency	Demonstrated
UI Test Automation	✓
Functional Testing	✓
Selenium WebDriver	✓
Python Programming	✓
PyTest Framework	✓
Page Object Model (POM)	✓
Explicit Waits	✓
Element Identification (Locators)	✓
Assertions	✓
Test Framework Design	✓
Automation Best Practices	✓
Test Documentation	✓

Project Screenshot

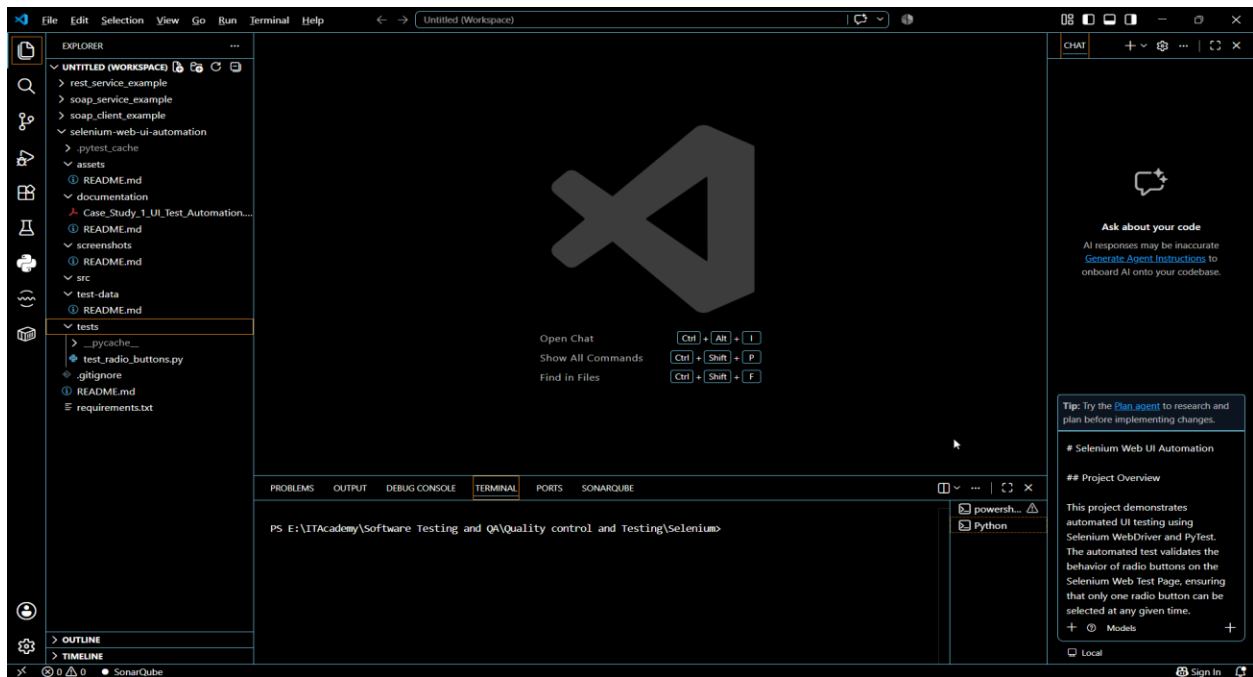


Figure 1 – project_structure

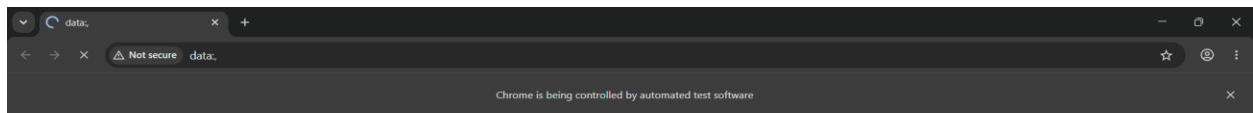


Figure 2 – browser_execution.1.png

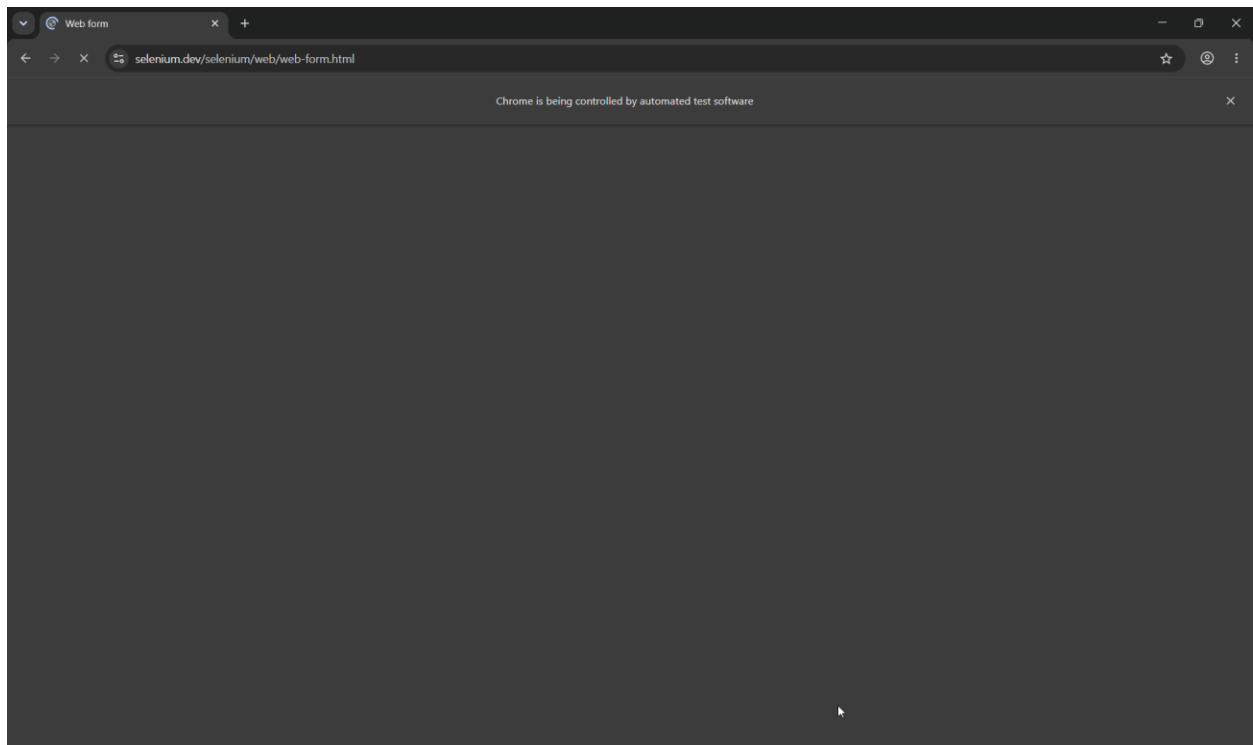


Figure 3 – browser_execution.2.png

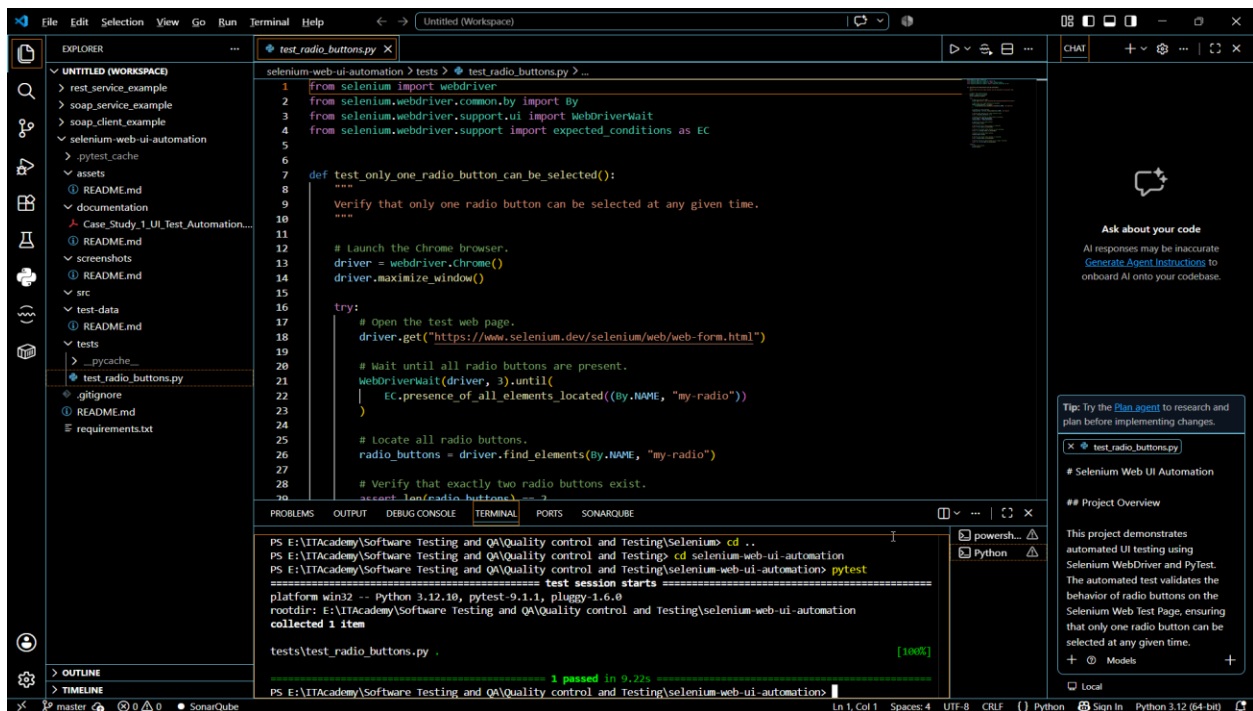


Figure 4 – pytest_execution.png

GitHub Repository

[View Selenium Automation Project on GitHub](#)

Recruiter Summary

This project demonstrates practical experience in designing and implementing a maintainable UI test automation framework using **Python**, **Selenium WebDriver** and **PyTest**. It highlights my ability to automate functional web application testing, apply the **Page Object Model (POM)** design pattern, implement reliable synchronization using **Explicit Waits**, and develop reusable, well-structured test scripts. The project reflects a systematic approach to test automation and reinforces my understanding of industry best practices for building scalable and maintainable automation frameworks.

The project demonstrates my readiness to contribute to UI test automation initiatives by developing reliable, reusable and maintainable automated testing solutions.
